

```
!pip install qiskit
```

```
Collecting qiskit
```

```
  Downloading qiskit-2.5.1-cp310-abi3-
```

```
manylinux_2_28_x86_64.whl.metadata (14 kB)
```

```
Requirement already satisfied: numpy<3,>=2.0 in
```

```
/usr/local/lib/python3.12/dist-packages (from qiskit) (2.0.2)
```

```
Requirement already satisfied: scipy>=1.14 in
```

```
/usr/local/lib/python3.12/dist-packages (from qiskit) (1.16.3)
```

```
Collecting rustworkx>=0.15.0 (from qiskit)
```

```
  Downloading rustworkx-0.18.1-cp310-abi3-
```

```
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (10 kB)
```

```
Requirement already satisfied: dill>=0.3 in
```

```
/usr/local/lib/python3.12/dist-packages (from qiskit) (0.3.8)
```

```
Collecting stevedore>=3.0.0 (from qiskit)
```

```
  Downloading stevedore-5.9.0-py3-none-any.whl.metadata (2.3 kB)
```

```
Requirement already satisfied: typing-extensions in
```

```
/usr/local/lib/python3.12/dist-packages (from qiskit) (4.16.0)
```

```
Downloading qiskit-2.5.1-cp310-abi3-manylinux_2_28_x86_64.whl (9.7 MB)
```

```
9.7/9.7 MB 60.8 MB/s eta
```

```
0:00:00
```

```
anylinux2014_x86_64.manylinux_2_17_x86_64.whl (2.4 MB)
```

```
2.4/2.4 MB 23.8 MB/s eta
```

```
0:00:00
```

```
54.5/54.5 kB 2.4 MB/s eta
```

```
0:00:00
```

```
from qiskit import QuantumCircuit
```

```
from qiskit.quantum_info import Statevector, Operator
```

```
from IPython.display import display
```

```
# Pauli-X Gate
```

```
qc_x = QuantumCircuit(1)
```

```
qc_x.x(0)
```

```
print("Pauli-X Gate Circuit:")
```

```
display(qc_x.draw())
```

```
print("Statevector:")
```

```
print(Statevector(qc_x))
```

```
Pauli-X Gate Circuit:
```

```
q: ── X ──
```

```
Statevector:
Statevector([0.+0.j, 1.+0.j],
            dims=(2,))
```

```
# Pauli-Y Gate
```

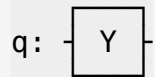
```
qc_y = QuantumCircuit(1)
```

```
qc_y.y(0)
```

```
print("Pauli-Y Gate Circuit:")
display(qc_y.draw())
```

```
print("Statevector:")
print(Statevector(qc_y))
```

```
Pauli-Y Gate Circuit:
```



```
Statevector:
Statevector([0.+0.j, 0.+1.j],
            dims=(2,))
```

```
# Pauli-Z Gate
```

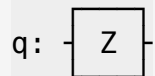
```
qc_z = QuantumCircuit(1)
```

```
qc_z.z(0)
```

```
print("Pauli-Z Gate Circuit:")
display(qc_z.draw())
```

```
print("Statevector:")
print(Statevector(qc_z))
```

```
Pauli-Z Gate Circuit:
```



```
Statevector:
Statevector([1.+0.j, 0.+0.j],
            dims=(2,))
```

```
# H followed by Z gate
```

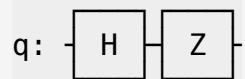
```
qc_hz = QuantumCircuit(1)
```

```
qc_hz.h(0)
qc_hz.z(0)

print("H-Z Circuit:")
display(qc_hz.draw())

print("Final State:")
print(Statevector(qc_hz))
```

H-Z Circuit:



```
Final State:
Statevector([ 0.70710678+0.j, -0.70710678+0.j],
            dims=(2,))
```

Five gate sequence

```
qc_random = QuantumCircuit(1)

qc_random.h(0)
qc_random.x(0)
qc_random.z(0)
qc_random.y(0)
qc_random.h(0)

print("Five Gate Circuit:")
display(qc_random.draw())

print("Final State:")
print(Statevector(qc_random))
```

Five Gate Circuit:



```
Final State:
Statevector([0.+1.j, 0.+0.j],
            dims=(2,))
```

Quantum Coin Flip

```
coin = QuantumCircuit(1)

coin.h(0)

print("Quantum Coin Flip Circuit:")
```

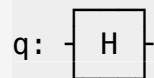
```
display(coin.draw())
```

```
state = Statevector(coin)
```

```
print("State:")
```

```
print(state)
```

Quantum Coin Flip Circuit:



State:

```
Statevector([0.70710678+0.j, 0.70710678+0.j],  
            dims=(2,))
```

```
import numpy as np
```

```
state = Statevector(coin)
```

```
probabilities = np.abs(state.data)**2
```

```
print("Heads Probability:", probabilities[0])
```

```
print("Tails Probability:", probabilities[1])
```

Heads Probability: 0.4999999999999999

Tails Probability: 0.4999999999999999

HZH circuit

```
hz_h = QuantumCircuit(1)
```

```
hz_h.h(0)
```

```
hz_h.z(0)
```

```
hz_h.h(0)
```

```
print("HZH Circuit:")
```

```
display(hz_h.draw())
```

```
print("HZH Matrix:")
```

```
print(Operator(hz_h))
```

HZH Circuit:



HZH Matrix:

```
Operator([[0.+0.j, 1.+0.j],
```

```
[1.+0.j, 0.+0.j]],  
input_dims=(2,), output_dims=(2,))
```

```
# X circuit
```

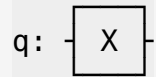
```
x_gate = QuantumCircuit(1)
```

```
x_gate.x(0)
```

```
print("X Circuit:")  
display(x_gate.draw())
```

```
print("X Matrix:")  
print(Operator(x_gate))
```

X Circuit:



X Matrix:
Operator([[0.+0.j, 1.+0.j],
[1.+0.j, 0.+0.j]],
input_dims=(2,), output_dims=(2,))

```
matrix1 = Operator(hz_h).data  
matrix2 = Operator(x_gate).data
```

```
if np.allclose(matrix1, matrix2):  
    print("HZH is equivalent to X gate")  
else:  
    print("HZH and X are different")
```

HZH is equivalent to X gate